Literature Review — Distributed Systems Lab

Optimized Task Offloading in IoT–Fog–Cloud Systems: A Comprehensive Literature Review of Metaheuristic Optimization, Deep Reinforcement Learning, SDN, and Mobility-Aware Computation Offloading

---

Author: Imed Eddine Benmadi

Keywords: Task offloading, IoT, Fog computing, NSGA-II, deep reinforcement learning, SDN, DAG task modeling, vehicular networks, mobility-aware offloading, proactive handoff

---

April 2026

Prepared for: Distributed Systems Lab — Literature Review Assignment

Abstract

The rapid growth of Internet of Things (IoT) deployments in smart-city and vehicular environments has intensified the demand for low-latency, energy-efficient computation at the network edge. Task offloading, the process of delegating compute-intensive work from resource-constrained IoT devices to fog or cloud infrastructure, has become the central mechanism for meeting these demands within the three-tier IoT–Fog–Cloud hierarchy.

This paper surveys ten representative works published between 2021 and 2025, spanning five research directions: multi-objective metaheuristic optimization using NSGA-II with Minimax Differential Evolution; task pre-filtering via the TOF-Broker Execution Cost framework; software-defined networking for programmable traffic management; microservice-oriented DAG task modeling; and mobility-aware offloading with deep reinforcement learning and proactive handoff prediction. For each direction, we identify the state of the art, its key contributions, and what it leaves unresolved.

From this analysis we extract five open research gaps. The two primary gaps concern integration and cold-start: no existing system simultaneously handles multi-objective optimization, task pre-filtering, DAG structure, online DRL adaptation, SDN routing, and mobility; and no system has validated offline evolutionary pre-training as a cold-start remedy in a full vehicular IoT context. We argue that bridging these gaps requires a hybrid architecture combining offline Pareto optimization with online reinforcement learning, applied to DAG workloads under GPS-driven mobility constraints.

Contents

1 Introduction **2**

2 Background and Formal Definitions **3**
2.1 Three-Tier IoT–Fog–Cloud Architecture . . . 3
2.2 Task Offloading Formulation . . . 3
2.3 Execution Cost and Task Classification . . . 3
2.4 Pareto Dominance . . . 3
2.5 DAG Task Model . . . 4

3 Research Evolution: Four Generations **4**
3.1 Generation 1 — Single-Objective Heuristics (before 2021) . . . 4
3.2 Generation 2 — Multi-Objective Evolutionary Algorithms (2021–2023) . . . 4
3.3 Generation 3 — Pre-Filtering and Hybrid Systems (2024) . . . 4
3.4 Generation 4 — Adaptive AI-Driven Systems (2024–2025) . . . 5

4 State of the Art **5**
4.1 Multi-Objective Optimization: NSGA-II with MMDE . . . 5
4.2 Task Pre-Filtering: TOF-Broker . . . 5
4.3 Software-Defined Networking for Fog Traffic . . . 6

4.4 Microservice Architecture and DAG Modeling 6
4.5 Deep Reinforcement Learning for Task Offloading 6
4.5.1 DRL in Vehicular Fog Networks 6
4.5.2 SDN-Integrated DRL: CSFD 7
4.5.3 DAG Offloading with Meta-RL and Offline Pre-training 7
4.6 Mobility-Aware Offloading 7
4.6.1 Reactive Handoff: Dual-Buffer System 7
4.6.2 Proactive Handoff: MAVTO 7
4.6.3 Spatiotemporal Prediction: Digital Twin with STGNN 8
4.7 Comparative Summary 8

5 Open Research Gaps 9

6 Proposed Architecture 10
6.1 Component Overview 10
6.2 Offline-to-Online Pipeline 11

7 Comparative Discussion 11

8 Conclusion 12

1 Introduction

The Internet of Things ecosystem is growing faster than most projections anticipated. Tens of billions of connected devices are expected to be active by 2030 [1], and a large share of them operate under conditions that traditional computing architectures were not designed for: they move, they generate data continuously, and the decisions they need support with are time-critical. A smart vehicle that needs to know whether a pedestrian is crossing its path cannot wait 200 ms for a response from a distant cloud server. An industrial robot operating on a millisecond control loop cannot tolerate network jitter. The latency introduced by conventional cloud computing, typically 50–200 ms due to wide-area propagation, makes it structurally unsuitable for these use cases [1].

Fog computing was introduced to close this gap by placing intermediate compute nodes one or two hops from the end device, bringing latency down to 1–10 ms. Together with the cloud, this creates the three-tier hierarchy that now defines the field: IoT devices at the bottom, fog nodes in the middle, cloud at the top. The fundamental question this hierarchy raises is *task offloading*: for each piece of computation arriving from an IoT device, which tier should execute it, under what constraints, and at what energy cost? This problem is formally NP-hard [11], and at the scale of real deployments no exact algorithm can solve it in time to be useful.

Five distinct challenges make task offloading hard in practice:

C1. Boulder problem. When compute-heavy tasks are routed to fog nodes without filtering, they consume the node's resources disproportionately and trigger queue cascade failures [2].

C2. Cold-start problem. Deep reinforcement learning agents begin with random weights and produce near-random decisions for thousands of steps before converging, which is unacceptable in deployed systems [6].

C3. Static optimization problem. Evolutionary algorithms like NSGA-II take hundreds of generations to converge, making them too slow for millisecond real-time decisions [8].

C4. Network congestion problem. Reactive SDN routing cannot respond to traffic bursts fast enough to prevent packet loss [3, 7].

C5. Mobility problem. In vehicular networks, devices cross fog coverage boundaries faster than tasks complete, causing mid-computation handoff failures [5, 9].

The central observation of this review is that no single published system resolves all five challenges together. Individual works advance one or two of them, sometimes at the expense of the others. The research field is mature enough that each challenge has been addressed in isolation; what is missing is a unified framework.

This paper is structured as follows. Section 2 establishes the formal definitions used throughout. Section 3 traces how the field has developed across four generations of work. Section 4 surveys the current state of the art in each sub-domain. Section 5 defines five open research gaps. Section 6 sketches the architecture that follows from closing them. Section 7 compares this architecture against related work. Section 8 concludes.

2 Background and Formal Definitions

2.1 Three-Tier IoT–Fog–Cloud Architecture

The three-tier model organizes computation into three layers. **Layer 1** (IoT devices) consists of resource-constrained task generators: smart vehicles, sensors, drones. They have limited CPU, battery, and memory. **Layer 2** (fog nodes) are edge servers co-located with 5G access points, offering low-latency compute within a coverage radius of roughly 150–300 m. **Layer 3** (cloud) provides high-throughput batch computation at the cost of 20–50 ms additional WAN delay.

2.2 Task Offloading Formulation

Let $\mathcal{T} = \{t_1, \ldots, t_N\}$ be a batch of N tasks from IoT devices. Each task t_i is described by:

$$t_i = \langle \ell_i,\ d_i,\ D_i,\ \mathcal{E}_i \rangle \tag{1}$$

where ℓ_i is computational demand in Million Instructions (MI), d_i is input data size in kilobytes, D_i is the end-to-end deadline in milliseconds, and $\mathcal{E}_i$ is device transmission energy in joules. The offloading decision assigns each task to a destination $x_i \in \{\mathrm{FOG}_1, \ldots, \mathrm{FOG}_K, \mathrm{CLOUD}\}$. The two optimization objectives are:

$$f_1(\mathbf{x}) = \sum_{i=1}^{N} E_i(x_i) \quad \text{(total energy, minimize)} \tag{2}$$

$$f_2(\mathbf{x}) = \sum_{i=1}^{N} L_i(x_i) \quad \text{(total latency, minimize)} \tag{3}$$

subject to deadline constraints $L_i(x_i) \leq D_i$ and capacity constraints $\sum_{i:\, x_i = k} \ell_i \leq C_k$, where C_k is the MIPS capacity of fog node k.

2.3 Execution Cost and Task Classification

The Execution Cost (EC) metric introduced in [2] measures whether a task is feasible on a fog node:

$$\mathrm{EC}(t_i) = \frac{\ell_i}{\text{fog_MIPS}} \quad \text{(seconds)} \tag{4}$$

A task is a *pebble* if $\mathrm{EC}(t_i) < \theta$ and a *boulder* if $\mathrm{EC}(t_i) \geq \theta$, where $\theta = 1.0\,\mathrm{s}$ is the standard threshold. Boulders occupy a fog node for over a second at full capacity, which is long enough to starve hundreds of concurrent lightweight tasks.

2.4 Pareto Dominance

Solution $\mathbf{x}^{(a)}$ dominates $\mathbf{x}^{(b)}$, written $\mathbf{x}^{(a)} \prec \mathbf{x}^{(b)}$, when:

$$f_k(\mathbf{x}^{(a)}) \leq f_k(\mathbf{x}^{(b)})\ \forall k, \quad \text{and} \quad f_j(\mathbf{x}^{(a)}) < f_j(\mathbf{x}^{(b)}) \text{ for some } j \tag{5}$$

The Pareto front $\mathcal{P}^*$ contains all non-dominated solutions. No element of $\mathcal{P}^*$ can improve on one objective without degrading another. NSGA-II approximates this front by evolving a population of candidate routing plans through selection, crossover, and mutation [1].

2.5 DAG Task Model

Modern IoT applications are not monolithic: they decompose into sequences of dependent processing steps [4]. The DAG model captures this as $G = (\mathcal{V}, \mathcal{E})$, where each node $v_i \in \mathcal{V}$ is a microservice step with demand $\langle \ell_i, d_i^{\text{in}}, d_i^{\text{out}} \rangle$ and each directed edge $(v_i, v_j) \in \mathcal{E}$ means v_j cannot start until v_i finishes [8]. End-to-end latency along the critical path is:

$$L_{\text{app}} = \sum_{v_i \in \text{critical path}} \left[T_{\text{exec}}(v_i) + T_{\text{transfer}}(v_i, v_{i+1}) \right] \tag{6}$$

where $T_{\text{transfer}} = 0$ when consecutive steps share the same node.

3 Research Evolution: Four Generations

3.1 Generation 1 — Single-Objective Heuristics (before 2021)

The earliest task offloading work treated the decision as binary: run locally or send to the cloud. Greedy algorithms, game theory, and single-objective metaheuristics such as Particle Swarm Optimization were common [1]. These methods optimized one metric at a time, operated on static topologies, and assumed devices stayed connected. The core limitation was that energy and latency pull in opposite directions. Routing everything to the cloud minimizes computation latency but maximizes transmission energy; running everything locally saves energy but misses deadlines. A useful solution requires balancing both simultaneously.

3.2 Generation 2 — Multi-Objective Evolutionary Algorithms (2021–2023)

Jafari and Rezvani [1] moved the field forward by framing fog offloading as a multi-objective problem and applying NSGA-II with Minimax Differential Evolution (MMDE) mutation. Instead of a single optimum, their approach produces a Pareto front of solutions that trade off energy against latency, giving operators the choice of operating point. The MMDE mutation operator:

$$\mathbf{V} = \mathbf{r}_1 + F \cdot (\mathbf{r}_2 - \mathbf{r}_3), \quad F = 0.5,\ \text{CR} = 0.9 \tag{7}$$

uses the difference between two population members to steer mutation toward productive regions rather than sampling blindly. The result converges faster and produces more robust solutions than standard polynomial mutation.

The remaining weakness is that the algorithm treats all tasks equally. Heavy tasks that would never realistically run on a fog node still consume optimization cycles, slowing convergence and occasionally crashing fog queues when the solution assigns them there.

3.3 Generation 3 — Pre-Filtering and Hybrid Systems (2024)

Nouri et al. [2] introduced the TOF-Broker, a classification layer that runs before NSGA-II. It computes EC (4) for each task and immediately routes boulders to the cloud, shrinking the search space by 15–30%. On a standard IoT workload this yields a 12.18% energy reduction and only 0.38% additional latency overhead. Concurrently, the microservices

community [4] demonstrated that modeling applications as DAGs enables partial offloading: lightweight steps stay on the fog while heavy steps go to the cloud, something monolithic task models cannot express.

The remaining problem is rigidity. Both the TOF-Broker and NSGA-II produce static routing decisions. When fog load shifts during a simulation run, or when a vehicle moves into a new zone mid-task, neither system can respond.

3.4 Generation 4 — Adaptive AI-Driven Systems (2024–2025)

The current generation replaces static decision-making with learned agents that update continuously. In vehicular fog settings, DRL-based offloading [6] showed that DQN agents outperform greedy policies under high load by spreading tasks more intelligently across fog nodes. CSFD [7] embedded a DRL agent inside the SDN controller to predict traffic bursts and install routing rules before they arrive. MAVTO [9] used vehicle GPS data to pre-position tasks at the next fog zone before the vehicle crossed the coverage boundary. Zhu et al. [8] combined all three ideas at the component level: DAG modeling, NSGA-II offline pre-training, and online meta-RL.

None of these systems addresses all five challenges at once, which is the gap this paper focuses on.

4 State of the Art

4.1 Multi-Objective Optimization: NSGA-II with MMDE

Jafari and Rezvani [1] remain the standard reference for multi-objective metaheuristic offloading. Their setup uses binary chromosomes encoding fog-or-cloud decisions per task, evaluated on the energy and latency objectives defined in equations (2) and (3). Non-dominated sorting produces Pareto fronts across generations; crowding distance maintains diversity within each front; and MMDE (7) replaces random mutations with directional ones drawn from the population's own spread. The practical consequence is that the algorithm discovers the energy–latency trade-off surface reliably and can be tuned to favor either objective by selecting a different point on the Pareto front.

What this work does not provide is any mechanism to handle tasks that are computationally infeasible for fog nodes. All tasks enter the optimization pipeline regardless of their cost, and the resulting solutions occasionally assign boulders to fog nodes, causing queue starvation for lighter tasks.

4.2 Task Pre-Filtering: TOF-Broker

Nouri et al. [2] address this directly. The TOF-Broker computes EC (4) for each incoming task before any optimizer runs and immediately separates boulders from pebbles. Boulders go to the cloud without entering the search space; only pebbles are handed to NSGA-II. This produces three concrete benefits: fog queue crashes disappear, NSGA-II converges faster on a smaller problem, and overall energy consumption drops by 12%. The 0.38% latency overhead is negligible in practice.

The open issue the TOF-Broker creates is what the authors call the pebble avalanche. When task arrival rates exceed roughly 1,000 per second, pebbles accumulate in the queue faster than NSGA-II can optimize them. The broker was designed to prevent fog crashes, but under extreme load it simply defers the crash rather than eliminating it.

4.3 Software-Defined Networking for Fog Traffic

Gasmi and Ezzedine [3] surveyed SDN-based approaches to fog offloading and established the theoretical case for decoupling routing intelligence from physical forwarding. In an SDN architecture, forwarding switches hold no routing state of their own; they execute rules installed by a centralized controller via the OpenFlow protocol. The controller sees the entire network at once and can reprogram all switches simultaneously with a single API call. For task offloading this matters because it enables VIP lanes: when a batch of pebbles needs to reach the cloud urgently, the controller can open a dedicated high-priority path across every intermediate switch in under 5 ms.

The limitation identified in the same survey is the controller bottleneck. When many devices request new flow rules simultaneously, the controller becomes overloaded, adding 8–15 ms latency per flow. This negates the advantage of SDN for latency-sensitive workloads and is the problem that motivates embedding an RL agent inside the controller.

4.4 Microservice Architecture and DAG Modeling

Li et al. [4] produced the most comprehensive recent survey of microservice-oriented resource management in fog and edge computing, covering 136 studies from 2020 to 2024. They organize the field around five pillars: service placement, resource provisioning, task scheduling and offloading, resource allocation, and replica selection. Their main finding is that while each pillar is reasonably well-studied individually, the interactions between them are almost entirely unexplored. Jointly optimizing across all five simultaneously, which is what a real system must do, has no published solution.

The DAG formalization used in this paper comes from Zhu et al. [8], who define application steps as nodes with per-step computational demands and data sizes, and inter-step dependencies as directed edges. This formulation enables partial offloading at step granularity and is the basis for the latency model in equation (6).

4.5 Deep Reinforcement Learning for Task Offloading

4.5.1 DRL in Vehicular Fog Networks

An anonymous study [6] is the closest published work to the compute-placement agent in our proposed architecture. It trains a DQN on a grid-city simulation where client vehicles offload tasks to nearby parked service vehicles. The agent observes queue depths, distances, and channel quality at all candidate nodes and selects a destination via epsilon-greedy policy with experience replay. The main finding is that DRL distributes load more evenly than greedy policies and avoids the congestion cascades that greedy methods suffer under high arrival rates.

Three limitations motivate our extensions: the system optimizes delay only with no energy term; it has no offline pre-training and therefore suffers cold-start degradation; and it assumes vehicles remain connected throughout task execution, ignoring handoff.

4.5.2 SDN-Integrated DRL: CSFD

The CSFD framework [7] embeds a DRL agent directly inside the SDN control plane. Rather than waiting for a packet to arrive and then consulting the controller for a routing rule (8–15 ms per flow), the agent predicts upcoming traffic bursts and pre-installs OpenFlow rules before the first packet appears. The result in a vehicular ad-hoc network evaluation is higher packet delivery ratio, lower latency, and lower controller CPU load compared to both reactive SDN and a DRL-only baseline without SDN.

CSFD is the direct prior work for the network-routing agent in our proposed architecture. Its limitation is that it addresses routing only. Deciding which fog node should compute a given task is handled by a separate, static mechanism that the paper does not describe or optimize.

4.5.3 DAG Offloading with Meta-RL and Offline Pre-training

Zhu et al. [8] is the architecturally closest published system to our proposal. They model each application as a DAG, run NSGA-II offline on historical batches to generate Pareto-optimal routing plans, convert the plans into labeled (state, action) pairs, and pre-train a Meta-RL agent via behavioral cloning. The meta-policy then adapts to new network environments quickly because it already has a sensible prior rather than starting from random weights. Their results show consistent improvement over both standalone DRL and standalone NSGA-II in dynamic network conditions.

The gaps in this work are the absence of mobility modeling and an SDN layer. The paper's future work section explicitly names both as needed extensions.

4.6 Mobility-Aware Offloading

4.6.1 Reactive Handoff: Dual-Buffer System

The collaborative offloading framework in [5] was the first to explicitly address mid-computation vehicle handoffs. When a vehicle disconnects from a fog node while a task is running, the framework moves the task from the normal task queue (NTB) to a high-priority handoff queue (HTB). The HTB preempts all other work and runs at maximum CPU priority until the task completes. The result is then held in memory, keyed to the vehicle's identifier, and pushed to the vehicle the moment it reconnects anywhere in the fog network. This eliminates re-submission and reduces end-to-end latency by over 50% in high-mobility scenarios compared to simply dropping and retransmitting.

The approach is entirely reactive: the system takes no action until the physical disconnection has already occurred. The time between when a disconnection becomes predictable and when it actually happens is wasted.

4.6.2 Proactive Handoff: MAVTO

MAVTO [9] introduced the first proactive mobility framework with a peer-reviewed evaluation on real vehicular datasets. Given a vehicle's GPS position, speed, and heading from its spatial telemetry, the system calculates when the vehicle will exit the current fog zone:

$$T_{\text{exit}} = \frac{R_{\text{fog}} - \text{dist}(v_{\text{pos}},\, f_{\text{pos}})}{v_{\text{closing}}} \tag{8}$$

where v_{closing} is the component of vehicle velocity directed toward the zone boundary. This value is compared against the estimated remaining execution time to select one of three operating modes: *direct* (task finishes before exit, do nothing), *predictive* (pre-spin a container on the destination fog node before the vehicle arrives), or *hybrid* (split computation between local and edge). Compared to non-mobility-aware baselines, MAVTO improves task completion success rate by more than 25%.

The mode selection is rule-based, not learned, and the system has no energy objective and no SDN routing for the container migration path.

4.6.3 Spatiotemporal Prediction: Digital Twin with STGNN

The most recent work in this space [10] combines a Digital Twin (a continuously synchronized virtual replica of the network), a Spatio-Temporal Graph Neural Network (STGNN) for demand forecasting, and A3C multi-agent DRL for task caching. The STGNN processes the vehicular network as a spatial graph with an LSTM temporal layer, predicting task demand per fog zone 5–10 seconds ahead and enabling proactive resource reservation. This is the current ceiling for trajectory prediction quality but lacks multi-objective optimization, task pre-filtering, and energy awareness.

4.7 Comparative Summary

Table 1 places all ten surveyed works across six capability dimensions. The proposed Predictive Cloud-Native Mobile Edge (PCNME) architecture is the only entry that checks all six.

Table 1: Surveyed works across six capability dimensions. ✓ = addressed, ○ = partial, × = not addressed.

Work	**Multi-obj.**	**Pre-filter**	**DAG**	**DRL**	**SDN**	**Mobility**	**Year**
Jafari & Rezvani [1]	✓	×	×	×	×	×	2021
Nouri et al. [2]	✓	✓	×	×	×	×	2024
Gasmi & Ezzedine [3]	×	×	×	×	✓	×	2023
Li et al. [4]	○	×	✓	○	×	×	2025
Zhu et al. [5]	×	×	×	×	×	○	2023
Toopchinezhad & Ahmadi [6]	×	×	×	✓	×	×	2024
Rahmatian et al. [7]	×	×	×	✓	✓	×	2025
Zhu et al. [8]	✓	×	✓	✓	×	×	2025
Zaidi et al. [9]	×	×	×	×	×	✓	2024
Zhu et al. [10]	×	×	○	✓	×	✓	2025
Proposed PCNME	✓	✓	✓	✓	✓	✓	2026

5 Open Research Gaps

The ten papers surveyed above converge on five gaps that none of them closes individually, and that no joint system in the literature has closed together. The first two are the primary contributions of this analysis.

Gap 1 — The Integration Gap. The most striking pattern in Table 1 is that every system addresses at most three of the six capability dimensions. This is not a coincidence of publication timing: it reflects a deeper structural difficulty. Compute placement decisions (which fog node handles a DAG step) and network routing decisions (which physical path carries that step's data) are interdependent in a way that current architectures do not account for. A placement that minimizes compute latency may route a task to a fog node whose upstream link is congested, adding more delay than it saves. Handling this correctly requires either a joint optimization over placement and routing simultaneously, which creates an intractable action space, or an architectural design where two specialized agents share the network-state information they each need. Neither approach has been demonstrated at the scale of DAG workloads with vehicle mobility.

Gap 2 — The Cold-Start Gap. Every DRL system surveyed initializes from random weights. This means the agent spends its early operational life making near-random

decisions, a period that spans thousands of interaction steps in typical training curves. For a vehicular safety system, near-random placement decisions during cold-start represent an operational hazard, not merely a performance cost. Zhu et al. [8] demonstrate that seeding DRL weights from NSGA-II Pareto solutions via behavioral cloning eliminates much of this degradation, but only in a setting without vehicle mobility and without SDN. Whether this pre-training advantage persists across the full complexity of a city-scale vehicular network, with both mobility and network management in play, remains an open question.

Gap 3 — The DAG-Mobility Gap. The mobility-aware systems reviewed in Section 4.6 treat tasks as indivisible units. When a task is instead a DAG with multiple concurrent steps running on different fog nodes, a vehicle handoff may invalidate some steps but not others depending on which fog node each step is assigned to. The T_{exit} comparison in MAVTO [9] is done once per task; it would need to be done once per DAG step, with the additional complexity that the relevant fog node differs per step. No published work addresses this.

Gap 4 — The Pebble Avalanche Gap. The TOF-Broker solves the boulder problem but exposes a new failure mode. When tasks arrive faster than roughly 1,000 per second, the pebble queue grows faster than NSGA-II can drain it. The queue eventually overflows, causing exactly the kind of cascade failure the broker was designed to prevent. A mechanism for graceful degradation in this regime, whether by dynamically lowering the EC threshold to reclassify heavy pebbles as boulders, or by batching overflow pebbles into super-tasks routed directly to cloud, has not been proposed or validated.

Gap 5 — The Trajectory Prediction Gap. MAVTO predicts vehicle positions using linear dead-reckoning, which works reasonably well on straight highway segments but degrades at intersections, during braking events, or when a vehicle takes an unexpected turn. Zhu et al. [10] demonstrate that a Spatio-Temporal Graph Neural Network trained on historical mobility data is substantially more accurate, but they apply it only to task caching, not to proactive handoff. A system that uses STGNN-quality trajectory prediction as the input to a proactive container migration mechanism, combined with Pareto-optimal placement, has not been built.

6 Proposed Architecture

The gaps identified above point toward a specific architecture. We call it the Predictive Cloud-Native Mobile Edge (PCNME) and describe its five components here. The intent is not to present a finished system but to show that the five gaps have a coherent joint solution.

6.1 Component Overview

TOF-Broker and Aggregator address Gaps 1 and 4. The broker applies EC classification (4) per DAG step before any optimizer runs. When the pebble queue exceeds a threshold, the aggregator bundles queued pebbles into a super-task and routes it to the cloud via an SDN VIP lane, preventing avalanche without losing the data.

TOF-MMDE-NSGA-II Offline Optimizer addresses Gap 2. The combination of TOF pre-filtering and MMDE-enhanced NSGA-II (7) generates Pareto-optimal routing plans from historical task logs. These plans are decomposed into (state, action) pairs for behavioral cloning of both DRL agents before the simulation begins.

DRL Agent 1 (task placement) addresses Gap 1. A Deep Q-Network pre-trained on NSGA-II Pareto data and fine-tuned online via experience replay selects which fog node handles each pebble DAG step. Its 13-dimensional state vector includes fog CPU loads, queue depths, task EC, bandwidth utilization, vehicle speed and heading, T_{exit}, remaining deadline, and cloud backlog. Actions are the four available fog nodes plus cloud.

DRL Agent 2 (SDN routing) addresses Gaps 1 and 4. A second DQN embedded in the SDN controller, pre-trained on historical city traffic patterns, predicts upcoming traffic bursts and pre-installs OpenFlow flow rules before the traffic arrives. It shares network-state telemetry with Agent 1, providing the coupling that resolves the interdependence identified in Gap 1.

Trajectory Predictor and NTB/HTB Buffers address Gaps 3 and 5. The predictor evaluates T_{exit} (8) separately for each DAG step rather than once per task, applying step-level proactive migration when $T_{\text{exec}} > T_{\text{exit}}$. The NTB/HTB dual-buffer system from [5] provides deterministic fallback when trajectory prediction fails.

6.2 Offline-to-Online Pipeline

The system operates in two phases. In the offline phase, historical task logs and mobility traces feed TOF-MMDE-NSGA-II. The resulting Pareto solutions are converted to labeled examples and used to initialize both agent networks via supervised learning. In the online phase, Agent 1 makes placement decisions in under 1 ms per pebble step; Agent 2 manages SDN rules proactively. Both update continuously from rewards of the form:

$$R = -0.5\, L_{\text{norm}} - 0.3\, E_{\text{norm}} - 0.2\, \mathbf{1}[L > D] \cdot \lambda \tag{9}$$

where $\lambda = 10$ for safety-critical steps and 1 otherwise. The offline-to-online knowledge transfer is what eliminates cold-start and is the central novelty of the design.

7 Comparative Discussion

Against Jafari and Rezvani [1]. The proposed system retains NSGA-II with MMDE as its offline optimizer, so it subsumes rather than replaces this work. The TOF-Broker reduces the problem NSGA-II has to solve; the DRL agents handle decisions NSGA-II is too slow to make in real time; and the mobility layer addresses vehicular dynamics that the original static setup cannot model.

Against Nouri et al. [2]. The TOF-Broker is incorporated unchanged. What the proposed system adds is MMDE mutation (faster convergence), two online DRL agents (real-time adaptation), the Aggregator (avalanche prevention), and GPS-driven mobility management. All four additions directly address limitations that Nouri et al. identify as future work.

Against Zhu et al. [8]. This is the closest prior work structurally. The main differences are the TOF pre-filter applied before NSGA-II runs, the SDN routing agent (Agent 2), and step-level proactive mobility. The future work section of [8] explicitly requests mobility support and SDN integration.

Against CSFD [7]. CSFD's DRL-in-SDN design is adopted directly as Agent 2. What CSFD does not provide is any compute placement intelligence. Agent 1 fills that role, and the shared telemetry between the two agents addresses the coupling Gap 1 identifies.

Against MAVTO [9]. The T_{exit} formula and three-mode framework are adopted from MAVTO. The proposed system extends this with step-level assessment, RL-learned mode selection replacing the rule-based threshold comparison, and SDN-managed migration paths. The NTB/HTB fallback from [5] handles cases where the trajectory prediction fails.

8 Conclusion

This paper reviewed ten works on task offloading in IoT–Fog–Cloud systems, tracing a clear development from single-objective greedy methods through multi-objective evolutionary algorithms, hybrid pre-filtering systems, and the current generation of adaptive AI-driven architectures. The review reveals that the field has reached a point where each individual challenge, the boulder problem, cold-start, static optimization, network congestion, and mobility, has a credible solution in isolation. What does not yet exist is a system that handles all of them together.

The two primary gaps, the Integration Gap and the Cold-Start Gap, define the most important open problem: whether a dual-agent architecture combining offline Pareto pre-training with online reinforcement learning can operate reliably at city scale with DAG workloads and vehicle mobility simultaneously in play. The evidence from the surveyed works suggests this is achievable by composition; what is needed is the engineering effort to combine the components and a rigorous empirical validation.

The proposed PCNME architecture is a concrete step in that direction. Its novelty is not in any individual component, each of which already exists in the literature, but in their integration and in the offline-to-online knowledge transfer mechanism that eliminates cold-start degradation before the system begins operating.

Future directions include replacing linear dead-reckoning with STGNN-based trajectory prediction [10], applying federated learning to train Agent 2 across multiple cities without sharing raw traffic data, and investigating TD3 or SAC as higher-performing replacements for DQN in the continuous-action resource allocation setting.

References

[1] V. Jafari and M. H. Rezvani, "Joint optimization of energy consumption and time delay in IoT–fog–cloud computing environments using NSGA-II metaheuristic algorithm,"

Journal of Ambient Intelligence and Humanized Computing, vol. 14, pp. 1675–1698, 2023 (published online 2021). https://link.springer.com/article/10.1007/s12652-021-03388-2

[2] N. Nouri and M. H. Rezvani, "A multi-objective approach for optimizing IoT applications offloading in fog–cloud environments with NSGA-II," *The Journal of Supercomputing*, 2024. https://link.springer.com/article/10.1007/s11227-024-06431-z

[3] K. Gasmi and T. Ezzedine, "Reinforcement learning based task offloading of IoT applications in fog computing: algorithms and optimization techniques," *Cluster Computing*, 2024. https://www.ijcnis.org/index.php/ijcnis/article/view/8458/2522

[4] Y. Li et al., "Energy-efficient resource management in microservices-based fog and edge computing: state-of-the-art and future directions," *ACM Computing Surveys*, 2025. https://arxiv.org/pdf/2512.04093

[5] M. Zhu, J. Liu, and H. Wang, "A collaborative offloading task framework for IoT fog computing," in *Proc. International Conference on Fog and Mobile Edge Computing*, 2023.

[6] M. P. Toopchinezhad and M. Ahmadi, "Deep reinforcement learning for delay-optimized task offloading in vehicular fog computing," *arXiv preprint arXiv:2410.03472*, 2024. https://arxiv.org/html/2410.03472

[7] M. Rahmatian, A. Javadpour, and F. Ja'fari, "CSFD: Enhancing congestion reduction in vehicular communication with SDN and fog integration using deep reinforcement learning," *Analog Integrated Circuits and Signal Processing*, 2025. https://link.springer.com/article/10.1007/s10470-025-02541-7

[8] L. Zhu, T. Liu, Y. Yang, and X. Luo, "Dynamic task offloading scheme for edge computing via meta-reinforcement learning," *Journal of Manufacturing Systems*, 2025. https://www.sciencedirect.com/science/article/pii/S1546221825000712

[9] S. Zaidi, M. A. Attalah, L. Khamer, and C. T. Calafate, "Task offloading optimization using PSO in fog computing for the internet of drones," *Drones*, vol. 8, no. 11, p. 696, 2024. https://www.mdpi.com/2504-446X/8/11/696

[10] L. Zhu et al., "A vehicular edge computing offloading and task caching solution based on spatiotemporal prediction," *Future Generation Computer Systems*, vol. 166, p. 107679, 2025. https://www.sciencedirect.com/article/abs/pii/S0167739X24006435

[11] I. Mashal et al., "Multiobjective offloading optimization in fog computing using deep reinforcement learning," *Journal of Computer Networks and Communications*, 2024.